

Exercices Lab 3 Bonus

1 Exercise

Write a function that takes “CryptographyIsFunAndIsDifferentThanCryptocurrency” as an argument and splits it in blocks of 16 bytes.

1.1 Approach

The algorithm performs the following steps:

1. The input string is first converted to a byte representation using UTF-8 encoding.
2. The required padding length is calculated based on the block size.
3. PKCS#7 padding bytes are appended to the data, where each padding byte's value equals the number of padding bytes added.
4. The padded byte string is then divided into 16-byte chunks.

1.2 Implementation

```
1 def split_into_byte_blocks(s: str, block_size: int = 16, encoding: str = "utf-8"):
2
3     # Convert the string to bytes
4     b = s.encode(encoding)
5
6     # Compute padding length
7     pad_len = block_size - (len(b) % block_size)
8     if pad_len == 0:
9         pad_len = block_size # always pad, even if multiple of block_size
10
11     # Apply PKCS#7 padding
12     b += bytes([pad_len]) * pad_len
13
14     # Split into blocks
15     blocks = [b[i:i+block_size] for i in range(0, len(b), block_size)]
16     return blocks
17
18 if __name__ == "__main__":
19     s = "CryptographyIsFunAndIsDifferentThanCryptocurrency"
20     blocks = split_into_byte_blocks(s, 16)
21     print(f"Original string ({len(s)} chars):\n{s}\n")
22     print(f"Total bytes (with padding): {len(b''.join(blocks))}\n")
23     for i, blk in enumerate(blocks, 1):
24         print(f"Block {i:02d}: {len(blk)} bytes | hex: {blk.hex()} | text: {blk.decode('utf-8', 'replace')}")
25
```

1.3 Output

The program divides the string into four 16-byte blocks. The last block is padded with 15 bytes (`\x0f`) to reach the 16-byte boundary:

```

Original string (49 chars):
CryptographyIsFunAndIsDifferentThanCryptocurrency

Total bytes (with padding): 64

Block 01: 16 bytes | hex: 4372797074667726170687949734675 | text: 'CryptographyIsFu'
Block 02: 16 bytes | hex: 6e416e644973446966666572656e7454 | text: 'nAndIsDifferentT'
Block 03: 16 bytes | hex: 68616e43727970746663757272656e63 | text: 'hanCryptocurrenc'
Block 04: 16 bytes | hex: 790f0f0f0f0f0f0f0f0f0f0f0f0f0f0f | text: 'y\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f\x0f'

```

2 Exercise

Using the PyCryptodome library, write a program that performs AES-CBC mode of encryption using only AES-ECB objects.

2.1 Overview

AES operates on 16-byte blocks, and the ECB mode simply encrypts each block independently. In contrast, CBC introduces **chaining** between blocks, which improves security by ensuring that identical plaintext blocks produce different ciphertexts.

In CBC mode, the encryption of each block C_i is defined as:

$$C_i = \text{AES_ECB_Encrypt}(P_i \oplus C_{i-1})$$

where P_i is the i -th plaintext block and C_{i-1} is the previous ciphertext block. For the first block, C_0 is replaced with the **Initialization Vector (IV)**. Decryption is performed using:

$$P_i = \text{AES_ECB_Decrypt}(C_i) \oplus C_{i-1}$$

Thus, by manually applying XOR operations and invoking the ECB encryption or decryption functions, we can simulate CBC behavior without using a dedicated CBC mode object.

2.2 Implementation Explanation

The implementation consists of several key components, each designed to emulate CBC behavior using only AES-ECB operations.

1. Padding and Unpadding. Before encryption, the plaintext is converted to bytes and padded using the **PKCS#7 padding scheme**. This ensures the length of the data is a multiple of the AES block size (16 bytes). Each padding byte contains a value equal to the number of padding bytes added. After decryption, these bytes are removed to restore the original message.

2. XOR Operation. A helper function performs a byte-wise XOR between two equal-length byte sequences. This is the fundamental operation that links blocks together in

CBC mode: each plaintext block is XORed with the previous ciphertext block (or the IV for the first block) before encryption.

3. AES-ECB Encryption. An AES cipher object is created in ECB mode using:

```
AES.new(key, AES.MODE_ECB)
```

This object exposes the `encrypt()` and `decrypt()` methods for processing individual 16-byte blocks. These are the only cryptographic operations used in the program.

4. CBC Encryption Process. For encryption, the algorithm processes the padded plaintext block by block:

1. XOR the current plaintext block with the previous ciphertext block (or IV).
2. Encrypt the resulting block using the AES-ECB object.
3. Store the output as the current ciphertext block.
4. Use this ciphertext block as the “previous block” for the next iteration.

The final ciphertext is obtained by concatenating all ciphertext blocks.

5. CBC Decryption Process. During decryption, the reverse steps are applied:

1. Decrypt each ciphertext block using the AES-ECB object.
2. XOR the decrypted block with the previous ciphertext block (or IV) to obtain the plaintext.
3. After processing all blocks, remove the PKCS#7 padding.

This process exactly reverses the encryption procedure, ensuring the original message is recovered.

6. Random Key and IV Generation. For every execution, the program generates a random 128-bit AES key and a 128-bit IV using PyCryptodome’s `get_random_bytes()` function. This guarantees that each encryption instance produces a unique ciphertext, even when the plaintext is the same.

2.3 Example Execution

For the message:

```
"N0tPres13y here"
```

The program generates a random 128-bit AES key and IV, then produces a unique ciphertext. When decrypted with the same key and IV, the original plaintext is recovered exactly.

```
Plaintext: b'NØtPresl3y here'
Key (hex): 930467937c7ecbae41d7481d05013ab7
IV (hex): fcb3ebceebeaa9b6bfe9af1e9cc24f96

Ciphertext (hex): e0bdf924e7cd9394719afc59069375c6
Ciphertext (base64): 4L35JOfNk5RxmvxZBpN1xg==

Decrypted plaintext: b'NØtPresl3y here'
Decryption successful.
```

3 Exercise

Simulate the nonce and key reuse attack in CTR mode using the PyCryptodome library

3.1 Core idea and mathematics

In CTR mode the block cipher is used to produce a keystream which is XORed with the plaintext:

$$CT = PT \oplus KS,$$

where KS is the keystream derived from $AES(key, nonce, counter)$. If the same key and nonce are reused for two different plaintexts P_1 and P_2 , the same keystream is re-used for both encryptions. Therefore

$$CT_1 \oplus CT_2 = (P_1 \oplus KS) \oplus (P_2 \oplus KS) = P_1 \oplus P_2.$$

This identity removes the keystream and reveals the XOR of the two plaintexts. From $P_1 \oplus P_2$ an attacker can:

- recover P_2 completely if P_1 is fully known (known-plaintext attack),
- recover prefixes of P_2 when lengths differ (only overlapping bytes),
- perform *crib-dragging* by guessing short likely fragments (cribs) and sliding them across $P_1 \oplus P_2$ to yield readable candidates for P_2 or P_1 .

3.2 How the script implements the attack

The script is intentionally minimalistic and proceeds as follows:

1. **Key and nonce.** It generates a random 128-bit key (`get_random_bytes(16)`) and an 8-byte nonce (`get_random_bytes(8)`), then (incorrectly) reuses the same pair for two encryptions.
2. **Encryption.** Two messages are encrypted with the same key and nonce: `c1 = ctr_enc(key, nonce, m.known)` and `c2 = ctr_enc(key, nonce, m.target)`. The script uses PyCryptodome's `AES.new(..., AES.MODE_CTR, nonce=nonce).encrypt`.
3. **Recovering plaintext.** The attacker computes $CT_1 \oplus CT_2$ (implemented by `xor(c1, c2)`), yielding $P_1 \oplus P_2$. Knowing P_1 , the script recovers the prefix of P_2 by XORing P_1 with $P_1 \oplus P_2$:

$$P_2 = (P_1 \oplus P_2) \oplus P_1.$$

The code does exactly that for the overlapping prefix.

4. **Crib-dragging helper.** The script performs a tiny automated crib-drag: it takes a short guess (the `crib` variable) and slides it across the $P_1 \oplus P_2$ stream; when XORing the crib with a slice yields printable ASCII, it reports a candidate—this emulates the manual crib-dragging technique attackers use to find plausible plaintext fragments.

The script prints the known message, the target, the recovered prefix, and whether the recovered prefix matches the target's prefix. The crib-drag loop is short and intended to quickly show how guesses can produce readable results.

3.3 Script

```

1 # Requires: pip install pycryptodome
2 from Crypto.Cipher import AES
3 from Crypto.Random import get_random_bytes
4
5 def xor(a,b): return bytes(x^y for x,y in zip(a,b))
6 def ctr_enc(key, nonce, pt): return AES.new(key, AES.MODE_CTR, nonce=nonce).encrypt(pt)
7
8 def main():
9     key = get_random_bytes(16)
10    nonce = get_random_bytes(8)
11
12    m_known = b"Thats right. Now, say my name"
13    m_target = b"You are Heisenberg"
14    #https://www.youtube.com/watch?v=dy_DAS7hDs
15
16    c1 = ctr_enc(key, nonce, m_known)
17    c2 = ctr_enc(key, nonce, m_target)
18
19    # attacker: ct1 ^ ct2 = pt1 ^ pt2
20    pt_xor = xor(c1, c2)
21    recovered = xor(pt_xor, m_known[:len(pt_xor)]) # recover prefix of m_target
22
23    print("known   :", m_known)
24    print("target  :", m_target)
25    print("recovered:", recovered)
26    print("full_recovery:", recovered == m_target[:len(recovered)])
27
28    # quick crib-drag (try a short guess)
29    crib = b"You are "
30    for pos in range(max(len(c1),len(c2)) - len(crib) + 1):
31        slice_xor = pt_xor[pos:pos+len(crib)]
32        if len(slice_xor) != len(crib): continue
33        cand = xor(slice_xor, crib)
34        if all(32<=ch<=126 for ch in cand):
35            print("crib match at", pos, "=>", cand)
36
37 if __name__ == "__main__":
38     main()

```

3.4 Output

```

known   : b'Thats right. Now, say my name'
target  : b'You are Heisenberg'
recovered: b'You are Heisenberg'
full_recovery: True
crib match at 0 => b'Thats ri'
crib match at 2 => b'M;grv;J-'
crib match at 8 => b'vbh}$Rh2'

```